



Assignment

Unit: Machine Learning, Artificial Intelligence And Algorithms
(PRT565)

Campus: ECP (Darwin)

Topic:

Customer Churn Prediction Using Logistic Regression, Decision Tree and Random

Submitted By

Md Adnan Abir (s382198)

Submitted To

Dr. Ms Yan Zhang

Lecturer, School of Engineering and Information Technology, CDU

Deadline: 31st Aug 2026

Table of Contents

List of Figures	3
1. Problem Description	4
2. Dataset Description	4
2.1 Correlation Check	6
3. Choice of Algorithm	8
3.1 Logistic Regression Model	8
3.2 Decision Tree Model	8
3.3 Random Forest Model	8
4. Description of Key Steps	9
4.1 Data Pre-processing and Cleaning	9
4.2 Data Scaling (Standardisation)	10
4.3 Splitting, Training and Testing	11
4.4 Additional Categorical Relationships	12
4.5 Hyperparameter Tuning and Overfitting Check	15
5. Results Obtained and Overall Understanding	16
5.1 Model Accuracy	16
5.2 Further Analysis and Understanding	19
References	21

List of Figures

Figure 1: Balance of the target variable (ChurnLabel).	6
Figure 2: Correlation heatmap of the numeric features.	7
Figure 3: Removing the data-leakage and identifier columns.	9
Figure 4: Categorical imputation using SimpleImputer.	9
Figure 5: Grouping the numeric tenure column into categories.	10
Figure 6: Label encoding and one-hot encoding of categorical data.	10
Figure 7: Splitting the data first, then scaling with StandardScaler.	11
Figure 8: Function used to train each model and report its metrics.	11
Figure 9: Churn based on contract type	12
Figure 10: Churn by internet connection type.	12
Figure 11: Churn by payment method.	13
Figure 12: Churn by paperless billing status.	13
Figure 13: Churn by tenure group (0 = 0-12 months, 3 = 49+ months).	14
Figure 14: Decision Tree training vs testing accuracy across depths.	15
Figure 15: Accuracy comparison of the three models	16
Figure 16: Confusion matrix for the Logistic Regression model.	17
Figure 17: Decision Tree (first two levels) using the entropy criterion.	18
Figure 18: F1-score (churn class) comparison across the three models.	18
Figure 19: Age distribution split by churn.	19
Figure 20: Monthly vs total charges, coloured by churn.	19

1. Problem Description

Trends in the industry show that losing customers is one of the most expensive problems for the phone business. People often say that getting a new customer can be 5–7 times more expensive than keeping an old one (Reichheld, 2001). This means that even a small drop in the rate of customer turnover can make a service a lot more money in the long run. Telecom companies lose 20 to 40 percent of their customers every year because the industry is stable and very competitive. This means that they need to find at-risk customers as soon as possible.

When a customer stops using a business's service, either by moving to a competitor's service or cancelling totally, this is called "churn." Based on a customer's traits, account information, and how much of a service they use, a provider may be able to predict which customers are likely to leave and offer them personalised incentives to stay with the company before they leave. In this study, I make and test several machine-learning models that can tell if a telecoms customer will leave (Yes) or stay (No). I also show how the data-analysis and preprocessing techniques we learned in this course work.

2. Dataset Description

The Telco client Churn data set contains 7,043 client profiles, each of which has up to 50 characteristics describing their profile, account and used services (Cortez, 2022). Of the 50 variables, 15 variables were removed and the remaining variables were selected as a basis for model creation and the target variable. The six columns were thrown out due to data leakage and nine were thrown out due to either being an identifier or a constant value.

Demographic attributes:

1. **Age:** age of the customers in years (19-80)
2. **Gender:** gender of the customer
 - Male:** male customer
 - Female:** female customer
3. **Married:** whether the customer is married (Yes / No)

- 4. Dependents:** whether the customer has dependents (Yes / No)
- 5. NumberofDependents:** number of dependents the customer has
- 6. SeniorCitizen:** whether the customer is a senior citizen (Yes / No)

Account attributes:

- 7. TenureinMonths:** number of months the customer has stayed with the company (1-72)
- 8. Contract:** the customer's contract term
 - Month-to-Month:** rolling monthly contract
 - One Year:** one year contract
 - Two Year:** two year contract
- 9. PaymentMethod:** how the customer pays (Bank Withdrawal / Credit Card / Mailed Check)
- 10. PaperlessBilling:** whether the customer uses paperless billing
- 11. Offer:** the last marketing offer accepted (Offer A–E or None)
- 12. MonthlyCharge:** the customer's current monthly charge in dollars (18.25–118.75)
- 13. TotalCharges:** total amount charged to the customer to date (18.8–8684.8)
- 14. TotalRevenue:** total revenue generated by the customer (21.36–11979.34)

Service attributes:

- 15. PhoneService:** whether the customer has phone service (Yes / No)
- 16. MultipleLines:** whether the customer has multiple phone lines (Yes / No)
- 17. InternetService:** whether the customer has internet service (Yes / No)
- 18. InternetType:** type of internet connection (DSL / Fiber Optic / Cable, or None)
- 19. OnlineSecurity, OnlineBackup, DeviceProtectionPlan, PremiumTechSupport:** add-on protection and support services (Yes / No)
- 20. StreamingTV, StreamingMovies, StreamingMusic:** entertainment streaming services (each Yes / No)
- 21. UnlimitedData:** whether the customer has an unlimited data plan (Yes / No)
- 22. AvgMonthlyGBDownload:** average monthly data download in GB (0–85)

Target variable:

23. ChurnLabel: output class — whether the customer churned

Yes (1): the customer left

No (0): the customer stayed

The dataset also includes post-outcome columns, such as ChurnScore, ChurnCategory, ChurnReason, CustomerStatus, SatisfactionScore and CLTV that describe or explain why the customer churned after the event has occurred. These identified as sources of data leakage were removed prior to modelling. The target is skewed, with approximately 27% of the customers churning. (Figure 1).

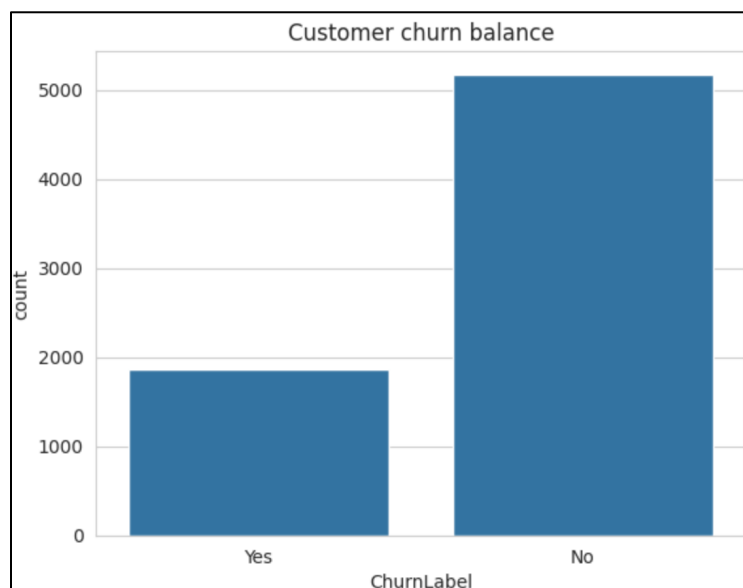


Figure 1: Balance of the target variable (ChurnLabel).

2.1 Correlation Check

The numerical features were tested to each other for correlation before modelling as the presence of two features with significantly similar information (multicollinearity) might corrupt the coefficients of a linear model and produce noise without contributing any predictive value. The number columns are correlated and are shown in a heatmap in Figure 2.

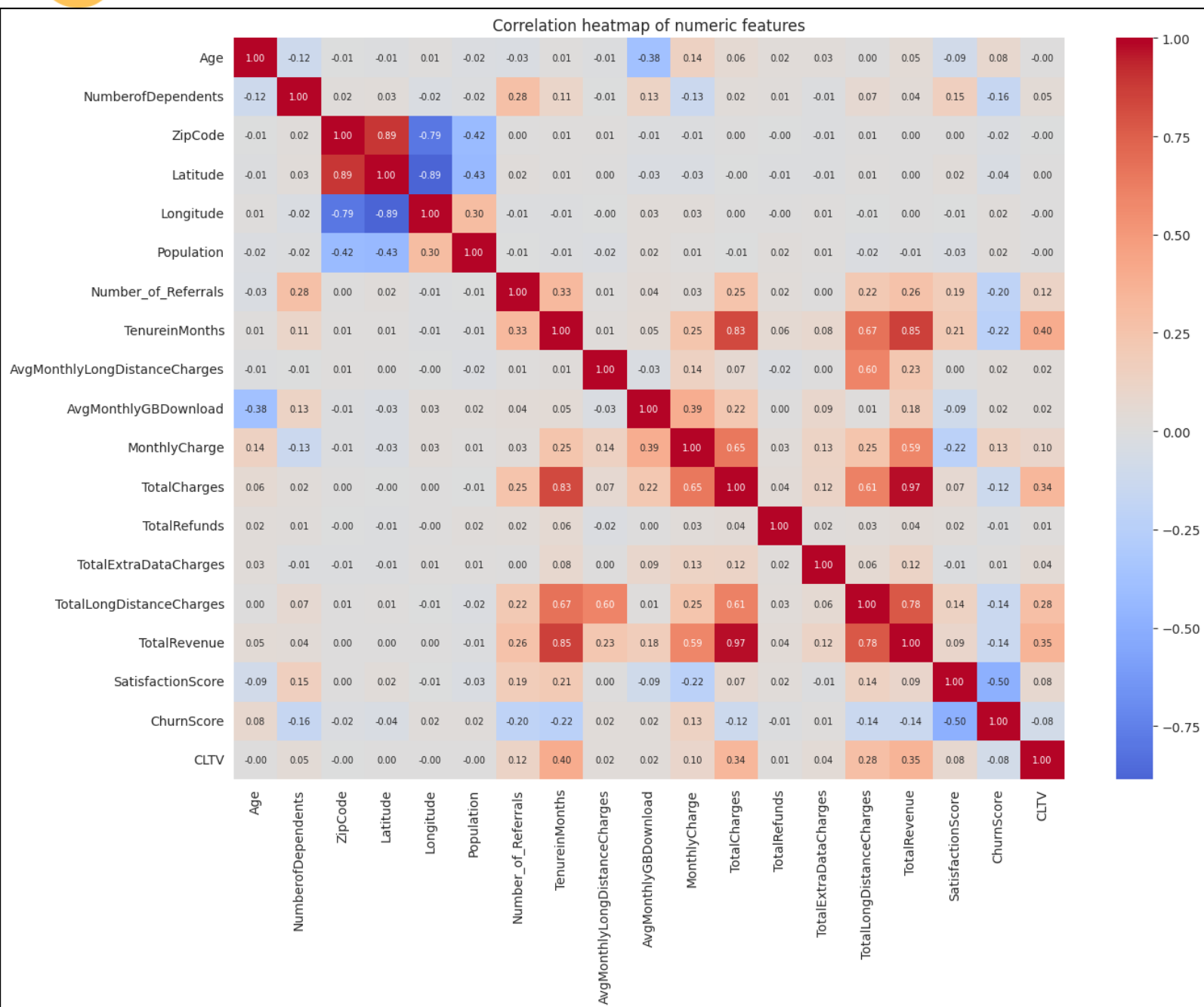


Figure 2: Correlation heatmap of the numeric features.

TotalCharges and TotalRevenue are both strongly correlated with TenureinMonths, which is expected since all three accumulate the longer a customer stays with the company. This was kept in mind when reviewing the Random Forest's feature importances later on, since correlated features can share credit for the same underlying signal rather than each being independently important.

Columns removed before modelling (15 of 50):

- **Data-leakage columns (6):** ChurnScore and ChurnCategory describe the churn outcome after the fact and CustomerStatus, SatisfactionScore including CLTV quantify the churn outcome after the fact. All of these would allow the answer to leak back into the model will lead to inaccurate result.
- **Identifier / constant columns (9):** CustomerID (a unique key), Country and State and Quarter (a single repeated value), City (over 1,100 distinct values), and ZipCode, Latitude, Longitude and Population (location fields) carry no general predictive value and were dropped.

3. Choice of Algorithm

As per recommendation three supervised classification algorithms were used so that different modelling approaches could be compared.

3.1 Logistic Regression Model

This algorithm gives a probability of churn and a threshold of 0.5 is used to classify. It is an efficient, interpretable linear model and gives a good baseline.

3.2 Decision Tree Model

A Decision Tree classifier splits the data into branches based on a series of questions. Data splitting was determined by the entropy criterion. It is very interpretable but a single tree overfits so its depth was limited.

3.3 Random Forest Model

A Random Forest is a collection of many individual trees of decisions and the class with the most votes across the trees is the model's prediction. It is usually more accurate and robust than a single tree and reports the most influential features.

4. Description of Key Steps

4.1 Data Pre-processing and Cleaning

There are a number of pre-processing operations that have been performed. The first were the six leaking columns in Section 2 that were removed. The reason for these columns (ChurnScore, ChurnCategory, ChurnReason, CustomerStatus, SatisfactionScore, and CLTV) being here is because of churn and their inclusion would see the result end up in the model and give an unrealistic accuracy. In addition, nine columns (CustomerID, Country, State, City, Quarter, ZipCode, Latitude, Longitude, Population) were dropped because they don't carry any predictive value.

Second, the categorical variables, Offer and InternetType, also contained some missing values (meaning customers did not have an offer or internet type). These were solved with the help of scikit-learn's SimpleImputer using a fixed value (Karim, 2025). Following this, there were no missing data points in the data set. Figures 3 and 4 show the programming for these two stages.

```
leakage      = ["ChurnScore", "ChurnCategory", "ChurnReason",
               "CustomerStatus", "SatisfactionScore", "CLTV"]
identifier   = ["CustomerID", "Country", "State", "City", "Quarter",
               "ZipCode", "Latitude", "Longitude", "Population"]
df.drop(columns=leakage + identifier, inplace=True)
print("Shape after dropping columns:", df.shape)
```

Figure 3: Removing the data-leakage and identifier columns.

```
for col in ["Offer", "InternetType"]:
    imputer = SimpleImputer(strategy="constant", fill_value="None")
    df[[col]] = imputer.fit_transform(df[[col]])

print("Total missing values remaining:", int(df.isnull().sum().sum()))
```

Figure 4: Categorical imputation using SimpleImputer

The numeric column TenureinMonths was binned into four lifecycle bands, as in the class approach with groups of a continuous variable (Figure 5). Afterwards, the categorical data was

encoded as binary yes/no and for the multi-category variables (Offer, InternetType, Contract, PaymentMethod), one-hot encoding was applied (Figure 6). The encoding resulted in a feature matrix with 42 predictor columns.

```
def tenure_group(months):
    if months <= 12: return 0
    elif months <= 24: return 1
    elif months <= 48: return 2
    else: return 3

df["TenureGroup"] = df["TenureinMonths"].apply(tenure_group)
df.drop(columns=["TenureinMonths"], inplace=True)
df["TenureGroup"].value_counts()
```

Figure 5: Grouping the numeric tenure column into categories.

```
# Encode the target: Yes-> 1, No-> 0
df["ChurnLabel"] = df["ChurnLabel"].map({"Yes": 1, "No": 0})

# Encode gender and all the Yes/No service columns to 1/0
df["Gender"] = df["Gender"].map({"Male": 1, "Female": 0})
yesno = ["Under30", "SeniorCitizen", "Married", "Dependents", "ReferredasFriend", "PhoneService",
         "MultipleLines", "InternetService", "OnlineSecurity", "OnlineBackup", "DeviceProtectionPlan",
         "PremiumTechSupport", "StreamingTV", "StreamingMovies", "StreamingMusic", "UnlimitedData",
         "PaperlessBilling"]
for col in yesno:
    df[col] = df[col].map({"Yes": 1, "No": 0})

# Onehot encode the remaining multicategory columns
df = pd.get_dummies(df, columns=["Offer", "InternetType", "Contract", "PaymentMethod"],
                    drop_first=True)

print("Feature matrix after encoding:", df.shape)
df.head()
```

Figure 6: Label encoding and one-hot encoding of categorical data.

4.2 Data Scaling (Standardisation)

The continuous numeric columns (Age, MonthlyCharge, TotalCharges and TotalRevenue) were transformed to be normally distributed with a mean of 0 and a standard deviation of 1 with the help of the StandardScaler. This is very important to avoid data leakage in Logistic Regression. The scaler was applied to the test set after it was created in the training set (Figure 7).

```

X = df.drop(columns=["ChurnLabel"])
y = df["ChurnLabel"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=9, stratify=y)

# scale the continuous numeric columns only
num_cols = ["Age", "NumberofDependents", "Number_of_Referrals", "AvgMonthlyLongDistanceCharges",
            "AvgMonthlyGBDownload", "MonthlyCharge", "TotalCharges", "TotalRefunds",
            "TotalExtraDataCharges", "TotalLongDistanceCharges", "TotalRevenue"]
scaler = StandardScaler()
X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols])

print("Training rows:", X_train.shape[0], " Testing rows:", X_test.shape[0])

```

Figure 7: Splitting the data first, then scaling with StandardScaler.

4.3 Splitting, Training and Testing

The data collection was divided into 75% training data and 25% testing data using `train_test_split`. Both sets had the same churn ratio by setting `stratify` to the objective, and a fixed `random_state` was used to make sure the results would be the same every time. Two more things were done to ensure no data was leaked: the split was done before to the scaling, and the `StandardScaler` was fitted exclusively on the training set before being used to scale the test set. This was to ensure that the data preparation for training was not influenced by the test customer data. Each of the three models was then trained on the training set using the `fit()` function and tested on the unseen test set using the `predict()` method (Figure 8). To quantify performance, I employed the F1-score, accuracy, precision, recall and confusion matrix for Logistic Regression.

```

def evaluate(name, model):
    model.fit(X_train, y_train)                # train based on the training set
    y_pred = model.predict(X_test)             # Predict based on the unseen test set
    print(name)
    print(" Accuracy : %.4f" % accuracy_score(y_test, y_pred))
    print(" Precision: %.4f" % precision_score(y_test, y_pred))
    print(" Recall   : %.4f" % recall_score(y_test, y_pred))
    print(" F1 score : %.4f" % f1_score(y_test, y_pred))
    return model, y_pred

```

Figure 8: Function used to train each model and report its metrics.

4.4 Additional Categorical Relationships

The same exploratory procedure used for contract type and three more category fields for instance kind of internet connection, the customer's payment method, and whether they use paperless billing, as these are often reported reasons of churn. Additionally, the tenure bands obtained after preprocessing were directly compared to churn.

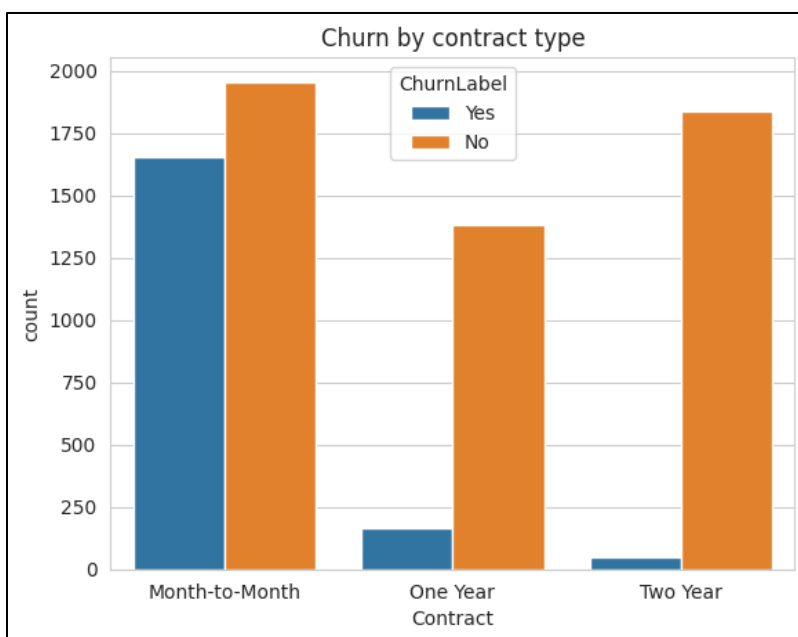


Figure 9: Churn based on contract type

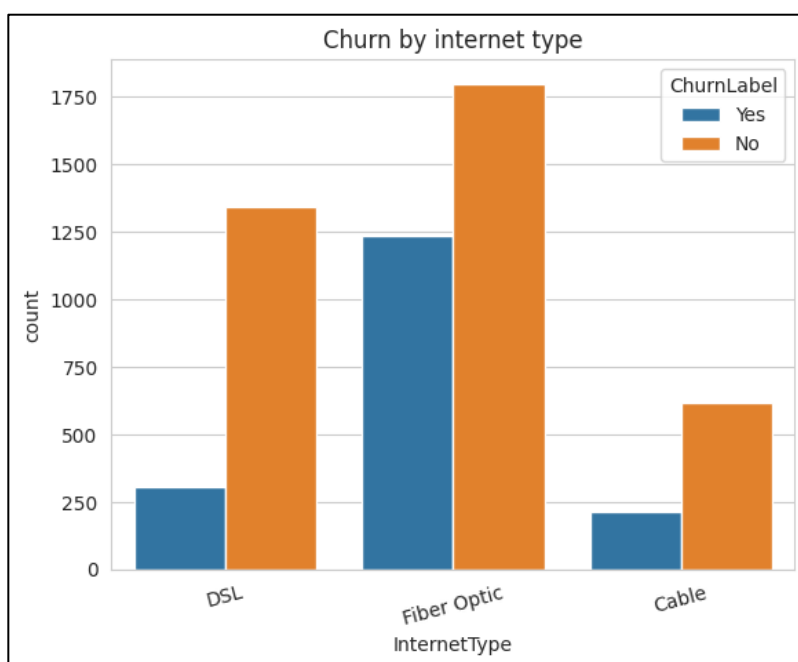


Figure 10: Churn by internet connection type.

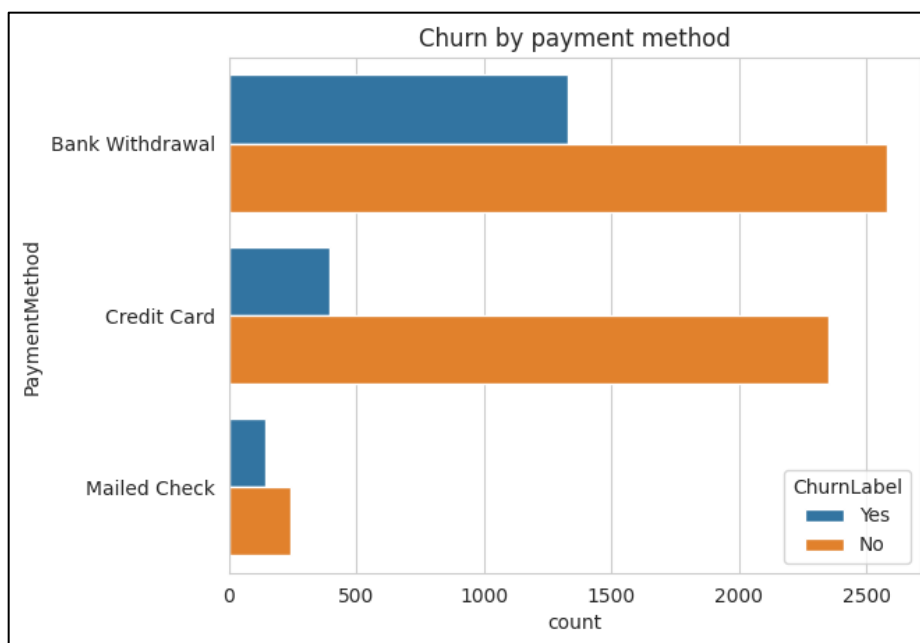


Figure 11: Churn by payment method.

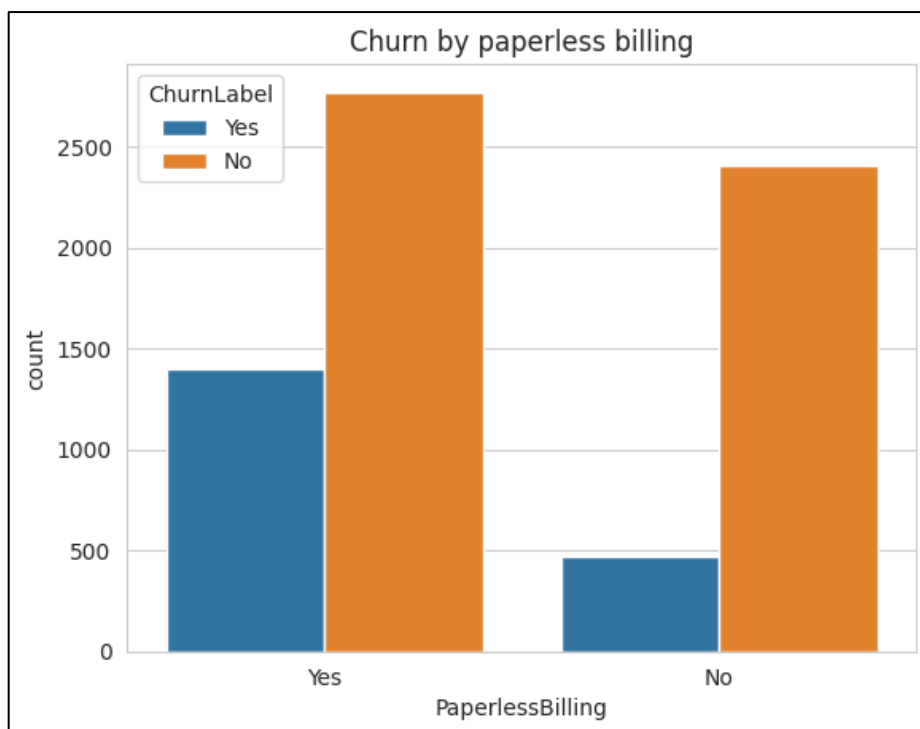


Figure 12: Churn by paperless billing status

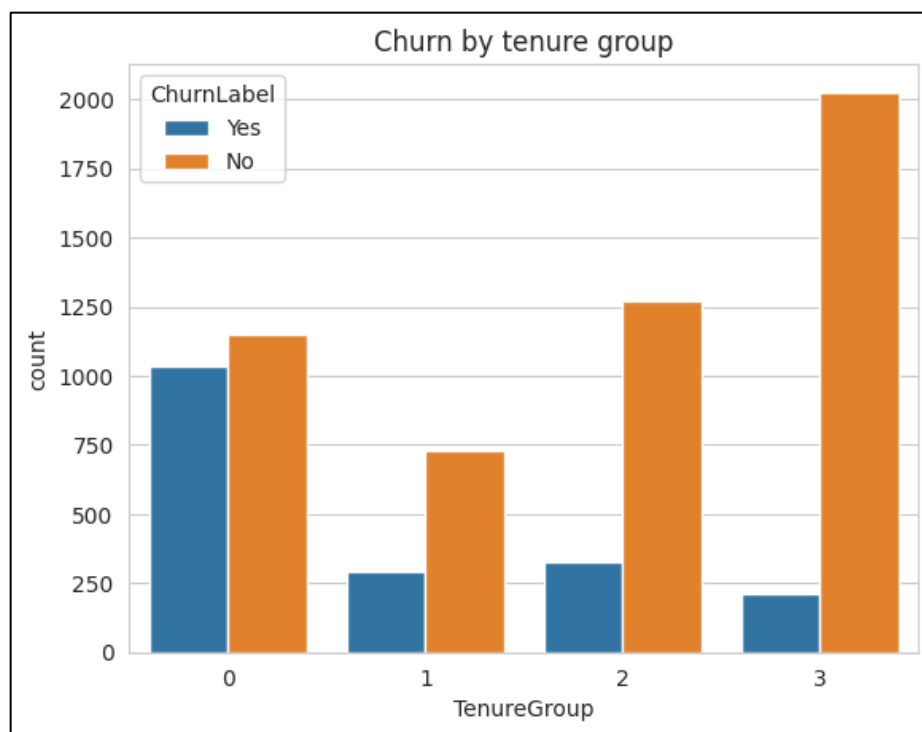


Figure 13: Churn by tenure group (0 = 0-12 months, 3 = 49+ months).

Customers with fibre-optic internet, electronic check payment and paperless billing have a much higher attrition rate than other groups. Churn is heavily concentrated in the shortest tenure band which is consistent with the month-to-month contract pattern.

4.5 Hyperparameter Tuning and Overfitting Check

A single Decision Tree can overfit if it is allowed to grow without limit, memorising the training data instead of learning a generalisable pattern. To check this, training and testing accuracy were compared across a range of tree depths.

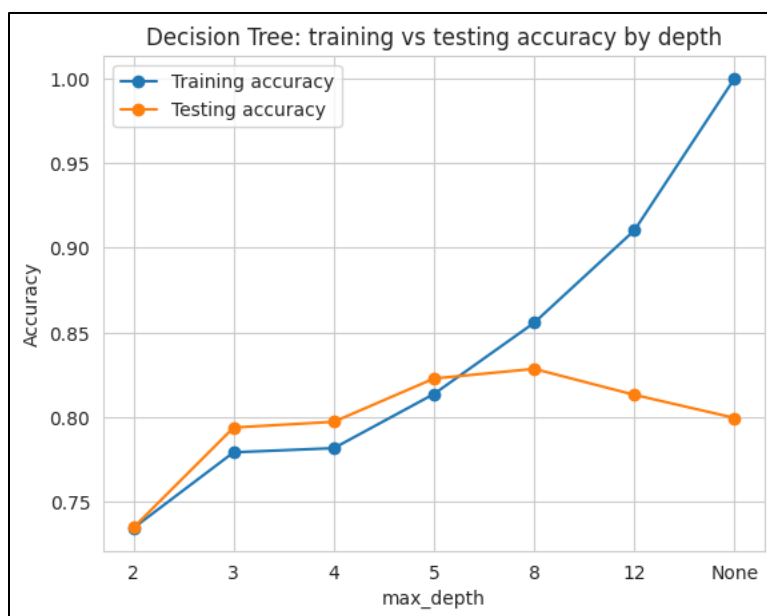


Figure 14: Decision Tree training vs testing accuracy across depths.

Overfitting is characterised by a flattening and subsequent mild drop in testing accuracy, whereas training accuracy approaches 1.0 as depth increases. This demonstrates that the narrow, limited tree used in Section 3.2 ($\text{max_depth} = 3$) generalises more well to unseen consumers than an unrestricted tree, despite the fact that the unrestricted tree seems more enticing based only on training data.

5. Results Obtained and Overall Understanding

5.1 Model Accuracy

The three models achieved the following accuracy on the unseen test set:

Model	Accuracy	F1 (churn)
Logistic Regression	0.846	0.69
Random Forest	0.846	0.66
Decision Tree	0.794	0.41

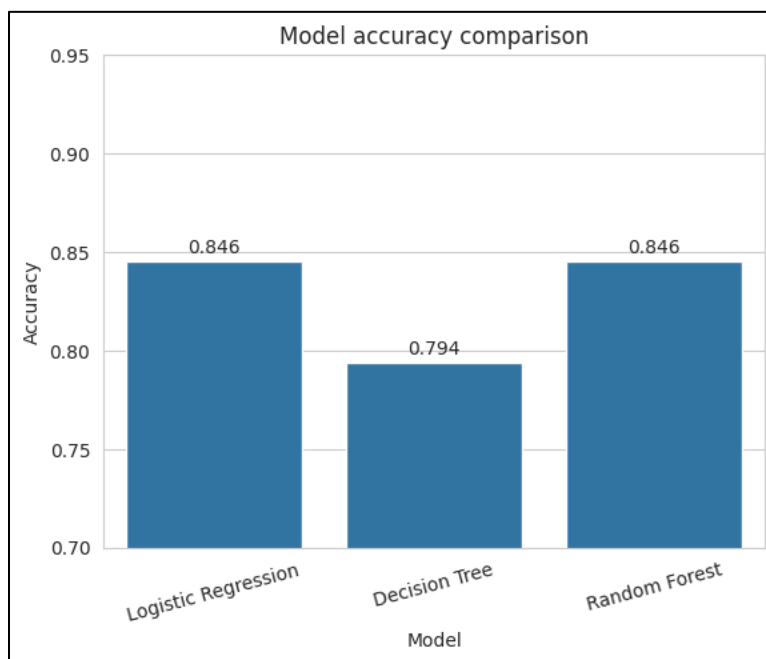


Figure 15: Accuracy comparison of the three models

The maximum accuracy (0.846) was attained with Logistic Regression and Random Forest, followed by the Decision Tree (0.794). Although Logistic Regression performs well generally in terms of accuracy, it is still misclassifying a proportion of true churners as “no churn”. This is seen via the confusion matrix. This is a direct consequence of the class imbalance and confirms that accuracy alone is not sufficient when dealing with imbalanced data; precision, recall and F1 give a more thorough overview. The Decision Tree was characterised by high accuracy and low recall meaning it was conservative in predicting churn and missed a large percentage of churners.

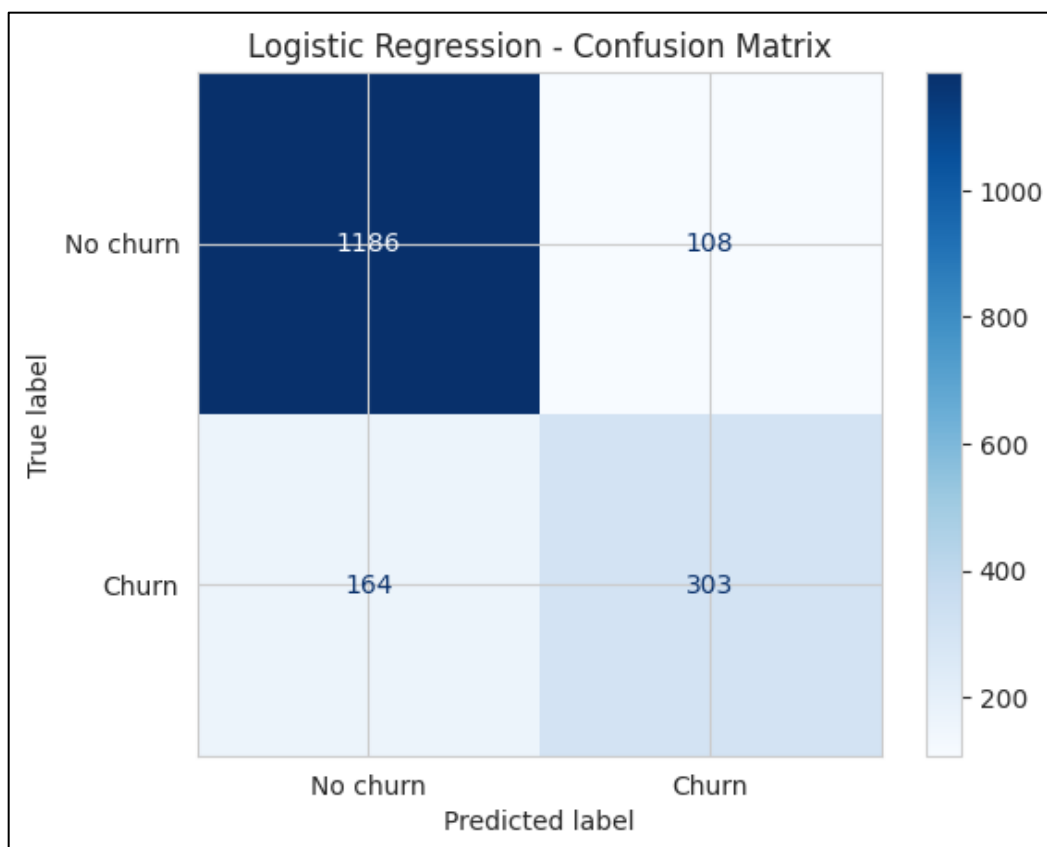


Figure 16: Confusion matrix for the Logistic Regression model.

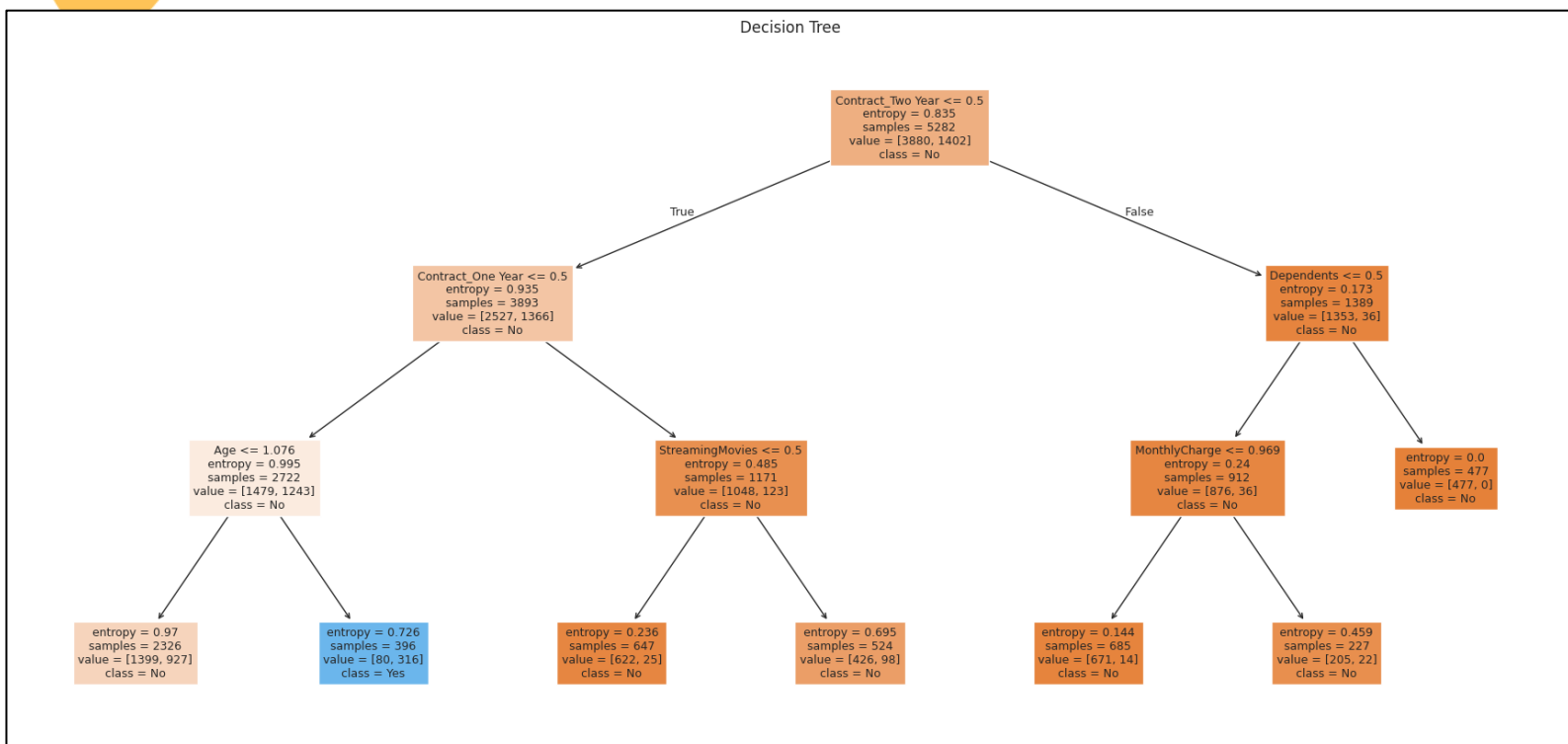


Figure 17: Decision Tree (first two levels) using the entropy criterion

Each model generated a complete classification report including accuracy, recall and F1 for both classes. Accuracy alone can be misleading on an imbalanced target (total churn $\sim 27\%$), thus further looked at the F1-score of the churn class for all three models.

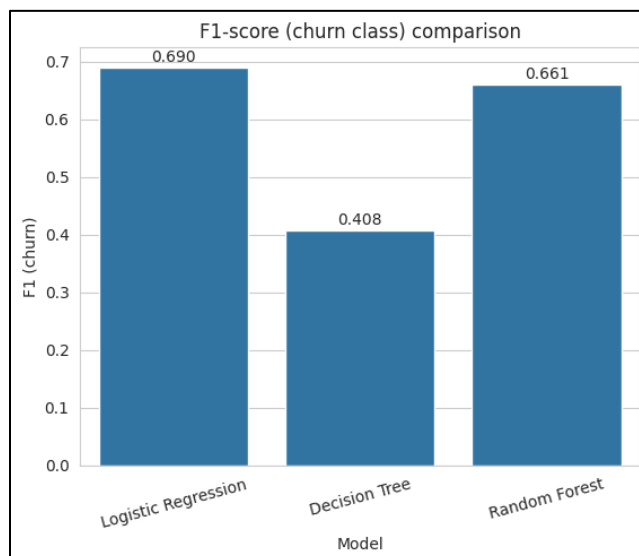


Figure 18: F1-score (churn class) comparison across the three models.

Logistic Regression scored the highest churn-class F1-score (0.69), followed closely by Random Forest (0.66) and Decision Tree (0.41). The low F1 score of the Decision Tree is mainly due to its

poor recall (0.27) as shown in the classification report despite a high precision (0.86). It rarely produces false positives, but misses around 75% of customers who actually churn which is an unfavourable trade-off for a business that wants to take action before customers churn.

5.2 Further Analysis and Understanding

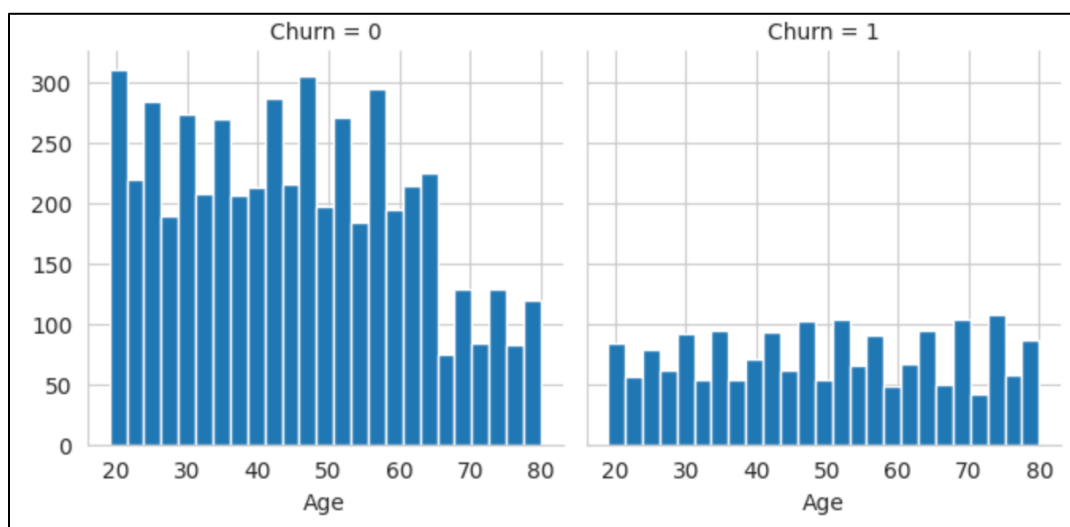


Figure 19: Age distribution split by churn.

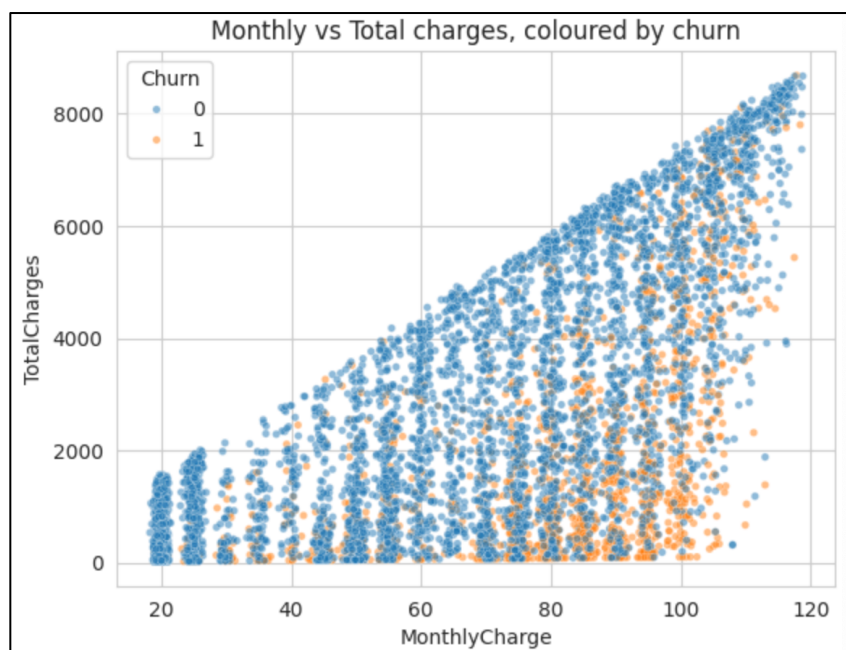


Figure 20: Monthly vs total charges, coloured by churn.

Exploratory charts show that customers with a month-to-month contract, a short term and high monthly prices are most likely to leave while customers with a long term rarely do. Finally, the full supervised learning process taught in the unit was activity demonstrated the. It included understanding and cleaning the data, removing leaks, imputing, binning, encoding, and scaling features, dividing the data into training and testing sets and properly rating three recommended models. The Random Forest showed the most significant reasons for churn while the Logistic Regression gave the most accurate and clear result. Getting rid of the columns that were leaking was the single most important thing that had to be done to reach a fair and reasonable result.

References

Cortez, R. R. (2022). *Telco customer churn* [Data set]. Kaggle.

<https://www.kaggle.com/datasets/rhonarosecortez/telco-customer-churn>

Karim, A. (2025). *PRT565 machine learning, artificial intelligence and algorithms: Weeks 3–5 lecture materials* [Lecture notes]. Charles Darwin University.

Reichheld, F. F. (2001). *The loyalty effect: The hidden force behind growth, profits, and lasting value*. Harvard Business School Press.